ARCHITECTURE GUIDE

Retrieval-Augmented Generation (RAG)

Enhancing Large Language Models with External Knowledge Sources

Overview

Retrieval-Augmented Generation (RAG) enhances the capabilities of Large Language Models (LLMs) by combining them with external knowledge sources. By dynamically retrieving relevant information from domain-specific or private datasets, RAG ensures accurate, up-to-date, and context-aware responses without requiring full model fine-tuning.

Core RAG Pipeline Components

A typical RAG system relies on seven foundational modular building blocks:

1. Document Loaders

Import raw data from PDFs, databases, web pages, and docs.

2. Text Splitters

Break documents into manageable, semantically coherent chunks.

3. Embedding Models

Convert text chunks into high-dimensional vector representations.

4. Vector Databases

Store and index embeddings for rapid semantic similarity search.

5. Retrievers

Locate and fetch the most relevant text chunks for a given query.

6. Prompt Templates

Combine retrieved context with user query into a unified prompt.

7. Large Language Models (LLMs)

Synthesize final, contextually grounded answers based on enriched prompts.

How RAG Works (Step-by-Step Flow)

Phase 1 — Ingestion

Documents are loaded and split into smaller, cohesive text chunks.

↓

Phase 2 — Vectorization

Each text chunk is converted into vector embeddings using an Embedding Model.

↓

Phase 3 — Storage

The generated embeddings are stored and indexed inside a specialized Vector Database.

↓

Phase 4 — Retrieval

When a user asks a question, the Retriever identifies and fetches the most relevant chunks.

↓

Phase 5 — Generation

Retrieved context chunks are passed to the LLM to generate an accurate, grounded, and context-aware response.